

ExpenseTracker: Core Functionality Implementation

Group Project Deliverable 2 (Saturday) - Option 1: Expense Tracker Application

Languages: Python and C++

Rahul Solanki, Krinal Soni, and Bijay Raj KC

University of the Cumberland

MSCS-632-M30: Advanced Programming Language

Jay Thom

October 3, 2026

ExpenseTracker: Core Functionality Implementation

Status

The core of ExpenseTracker is implemented and operational in both languages. Each stores a seeded set of nine expenses with a date, an amount, a category and a description, lists them oldest first, filters them by category through an index, and reports totals by category and overall. Both report the same figures. Date-range filtering, keyword search, percentage shares, deletion, the interactive prompt, the benchmark and the automated tests are scheduled for Sunday.

Table 1

Requirement Status at the End of Day 2

Requirement	Status
Store expenses with date, amount, category and description	Complete
List every expense, oldest first	Complete
Filter by category using an index rather than a scan	Complete
Totals by category and overall	Complete
Date-range filter and keyword search	Day 3
Percentage shares, delete, interactive prompt, benchmark	Day 3
Unit tests	Day 3

Implementation

Both programs follow the two-layer design agreed on Friday: a store owns the records and answers queries, and a driver prints the results. The difference is visible in the first ten lines of each file. Python holds each expense as a dictionary inside a dictionary keyed by id, with a defaultdict mapping each category to a list of ids. C++ declares a struct whose fields are fixed at compile time, stores the records in a `std::vector` and indexes them with a `std::unordered_map`. Both indexes are hash tables, so the two programs do the same algorithmic work.

The second difference is in how a value enters the system. The Python store converts the amount with `float()`, which accepts an int, a float or the string "34.25" without complaint, and the

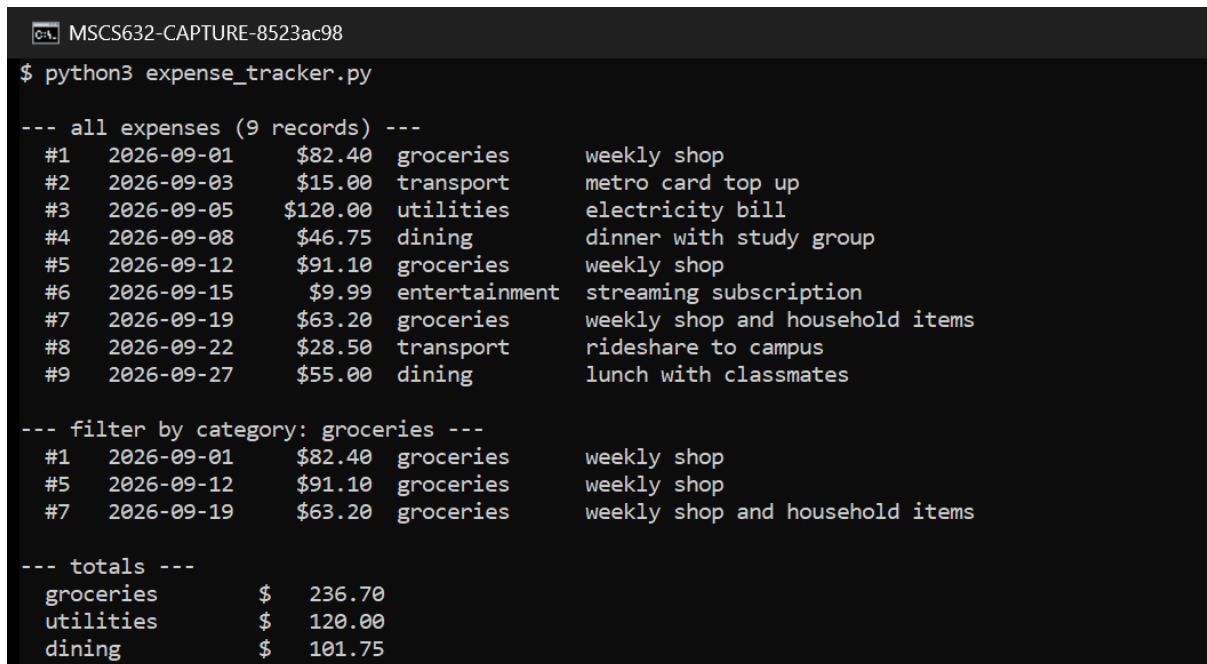
demo ends by adding exactly such a string to show that it works. The C++ signature declares the parameter as a double, so the equivalent call is rejected by the compiler and never reaches a test. Dates show the same split: Python parses with `datetime.strptime` from the standard library, while the C++ version has a hand-written `parse_date` because C++20 supplies a `calendar` type but not a parser in this toolchain.

Verification

Both implementations were built and run in WSL2 Ubuntu 24.04 using Python 3.12.3 and g++ 13.3.0, so a single machine underlies every result. The C++ build is clean under `-Wall -Wextra`. Each program reports nine records totalling \$511.94, and \$546.19 after the tenth expense is added, with identical per-category totals.

Figure 1

Python Core Implementation Running



```

C:\ MSCS632-CAPTURE-8523ac98
$ python3 expense_tracker.py

--- all expenses (9 records) ---
#1  2026-09-01    $82.40 groceries    weekly shop
#2  2026-09-03    $15.00 transport   metro card top up
#3  2026-09-05   $120.00 utilities   electricity bill
#4  2026-09-08    $46.75 dining      dinner with study group
#5  2026-09-12   $91.10 groceries    weekly shop
#6  2026-09-15     $9.99 entertainment streaming subscription
#7  2026-09-19   $63.20 groceries    weekly shop and household items
#8  2026-09-22   $28.50 transport   rideshare to campus
#9  2026-09-27   $55.00 dining      lunch with classmates

--- filter by category: groceries ---
#1  2026-09-01    $82.40 groceries    weekly shop
#5  2026-09-12   $91.10 groceries    weekly shop
#7  2026-09-19   $63.20 groceries    weekly shop and household items

--- totals ---
groceries    $   236.70
utilities    $   120.00
dining       $   101.75

```

Note. The final line adds an amount supplied as the string "34.25", which Python accepts silently. The equivalent call does not compile in C++.

Figure 2*C++ Core Implementation Running*

```

C++ MSCS632-CAPTURE-72ebaa86
$ g++ -std=c++20 -O2 expense_tracker.cpp && ./expense_tracker

--- all expenses (9 records) ---
#1  2026-09-01    $82.40  groceries    weekly shop
#2  2026-09-03    $15.00  transport   metro card top up
#3  2026-09-05   $120.00  utilities   electricity bill
#4  2026-09-08    $46.75  dining      dinner with study group
#5  2026-09-12    $91.10  groceries    weekly shop
#6  2026-09-15     $9.99  entertainment streaming subscription
#7  2026-09-19    $63.20  groceries    weekly shop and household items
#8  2026-09-22    $28.50  transport   rideshare to campus
#9  2026-09-27    $55.00  dining      lunch with classmates

--- filter by category: groceries ---
#1  2026-09-01    $82.40  groceries    weekly shop
#5  2026-09-12    $91.10  groceries    weekly shop
#7  2026-09-19    $63.20  groceries    weekly shop and household items

--- totals ---
groceries    $236.70
utilities    $120.00
dining       $101.75

```

Note. The same nine records and the same totals, produced from a `std::vector` of structs indexed by a `std::unordered_map`.

Code Quality and Testing

Both files are organised the same way, with the record type first, then the store, then the driver, and each language-specific decision is commented where it occurs rather than only in this report. Testing at this stage was manual: confirming that the listing is ordered oldest first, that the category filter returns the three grocery records, and that the per-category totals sum to the overall total in both programs. Automated tests and a sanitizer run are part of Sunday's deliverable.

Issues Found and Resolved

Two problems surfaced during the day. First, the C++ store indexes records by their position in the vector, and pushing a record while holding a reference to the vector's contents is unsafe once the vector reallocates. We reserved capacity up front and took the category string

before the move, which resolved it; the deeper consequence for deletion is deferred to Day 3. Second, category matching initially differed between the two programs because Python lower-cased the category on entry and C++ did not, so a search for "Groceries" returned three records in one program and none in the other. Both now normalise the category when the record is created.

Repository

The code, the README with build and run instructions, and these screenshots are in the repository at <https://github.com/iknalos/MSCS-632-M30-Group-D2>. The README documents the layout, the exact toolchain versions, the commands to build and run each implementation, and the language-specific features demonstrated in each file.

References

- Python Software Foundation. (2026). *The Python language reference* (Version 3.14). Retrieved October 3, 2026, from <https://docs.python.org/3/reference/>
- Stroustrup, B. (2013). *The C++ programming language* (4th ed.). Addison-Wesley.